

Virtualização

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

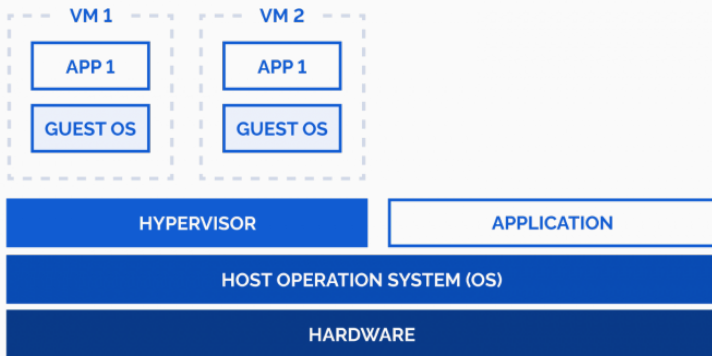
Introdução

O que é a Virtualização? i

A **Virtualização** cria uma versão baseada em software, ou “virtual”, de um computador. Esta Máquina Virtual (VM) corre como uma aplicação no seu computador físico, mas comporta-se como uma máquina completamente separada.

- **Anfitrião (Host):** A sua máquina física e o seu Sistema Operativo (SO).
- **Convidado (Guest):** A máquina virtual e o SO que ela executa.
- **Hypervisor:** O software que cria e gere as VMs.

O que é a Virtualização? ii



O Desafio: Instruções Privilegiadas

Uma aplicação normal não pode aceder diretamente ao hardware; tem de pedir ao SO Anfitrião. Mas um SO Convidado *espera* ter controlo total. Como resolvemos este conflito de forma segura?

O principal trabalho do hipervisor é intercetar e gerir de forma segura os pedidos do convidado para acesso privilegiado ao hardware. A forma como o faz define a diferença entre emulação e virtualização.

Tipos de Virtualização

Definição: A emulação envolve o uso de software para imitar o hardware de um sistema *diferente*. O hipervisor atua como um tradutor, convertendo as instruções da arquitetura da CPU do convidado para a arquitetura da CPU do anfitrião.

Caso de Uso: Executar um videojogo clássico concebido para uma consola baseada em ARM (como a Nintendo Switch) ou uma consola baseada em PowerPC (como a GameCube) no seu PC x86. O emulador (p. ex., Yuzu ou Dolphin) traduz o código do jogo em tempo real.

Emulação: O Caminho de uma Instrução

O hipervisor (emulador) tem de inspecionar e traduzir cada instrução em software antes que esta possa ser executada pelo hardware do anfitrião.

Vantagens

- **Compatibilidade entre Arquiteturas:** É a sua maior força. Permite que software desenhado para um tipo de CPU (p. ex., ARM) corra num tipo completamente diferente (p. ex., x86).

Desvantagens

- **Muito Lenta:** O passo de tradução de software para cada instrução cria uma sobrecarga de desempenho significativa, tornando-a muito mais lenta do que a execução de código nativo.
- **Elevado Uso de Recursos:** O processo de tradução em si é computacionalmente caro e consome muitos ciclos de CPU do anfitrião.

Virtualização Completa: Definição e Caso de Uso

Definição: A Virtualização Completa executa um SO convidado *não modificado* num ambiente de hardware simulado que corresponde à arquitetura do anfitrião. Baseia-se em **assistência de hardware da CPU** (Intel VT-x / AMD-V) para executar o código de forma eficiente. O SO convidado não tem consciência de que está a ser virtualizado.

Caso de Uso: Um utilizador de macOS a executar uma versão completa do Windows 11 no VirtualBox para usar um software específico que não está disponível para macOS, como um programa de CAD ou um jogo de PC específico.

Virtualização Completa: O Caminho de uma Instrução

As instruções não privilegiadas são executadas diretamente na CPU do anfitrião a toda a velocidade. Quando o convidado tenta executar uma instrução privilegiada, o hardware da CPU automaticamente a **intercepta ("trap")** e entrega o controlo de forma transparente ao hipervisor para que este a trate de forma segura.

Virtualização Completa: Vantagens e Desvantagens

Vantagens

- **Elevada Compatibilidade:** Pode executar qualquer sistema operativo padrão sem modificações.
- **Bom Desempenho:** A assistência de hardware torna-a significativamente mais rápida do que a emulação.
- **Forte Isolamento:** Os convidados estão isolados de forma segura do anfitrião e uns dos outros pelo hardware.

Desvantagens

- **Sobrecarga do “Trap”:** O ciclo “trap-and-emulate” para instruções privilegiadas ainda introduz alguma sobrecarga de desempenho, que pode ser significativa em cargas de trabalho intensivas em I/O (Entrada/Saída).

Paravirtualização: Definição e Caso de Uso

Definição: Na Paravirtualização, o SO convidado está *ciente* de que é uma VM e foi modificado com drivers especiais. Em vez de realizar ações que seriam interceptadas, comunica diretamente com o hipervisor através de uma API eficiente.

Caso de Uso: Esta é a base da computação em nuvem moderna. Um servidor web de alto desempenho a correr numa instância EC2 da Amazon Web Services (AWS) usa drivers paravirtualizados **VirtIO** para os seus dispositivos de disco e rede, para maximizar o débito e a baixa latência.

Paravirtualização: O Caminho de uma Instrução

O SO convidado sabe que não pode aceder diretamente ao hardware, por isso o seu driver “consciente” faz uma **“Hypercall”** — uma chamada de função direta e altamente eficiente ao hipervisor, evitando completamente o mecanismo de “trap”.

Paravirtualização: Vantagens e Desvantagens

Vantagens

- **Desempenho Mais Elevado:** Ao evitar a sobrecarga do “trap”, oferece o melhor desempenho, especialmente para tarefas intensivas de disco e rede.
- **Eficiente:** Menor sobrecarga de CPU em comparação com a virtualização completa, porque o caminho de comunicação é otimizado.

Desvantagens

- **Requer Modificação do SO Convidado:** Não pode executar um SO padrão não modificado. O SO precisa de ter os drivers de paravirtualização específicos instalados (embora a maioria das versões modernas de Linux e Windows já os inclua).

Resumo Comparativo

Característica	Emulação	Virtualização Completa	Paravirtualização
Conceito Central	Imitar hardware diferente	Isolar um SO não modificado	Cooperar com um SO consciente
Desempenho	Muito Baixo	Bom	Excelente
Modificação do SO	Não	Não	Sim
Hardware	Qualquer guest em qualquer host	Mesma arquitetura	Mesma arquitetura
Mecanismo	Tradução por Software	Trap & Emulate por HW	Hypercalls
Caso de Uso	Jogos Retro, Dev Cross-Arch	Desktop, Sistemas Legados	Cloud, Data Centers

Casos de Uso

A virtualização é a espinha dorsal da nuvem moderna.

- **Consolidação de Servidores:** Um único servidor físico potente pode substituir dezenas de servidores mais antigos, executando cada um como uma VM separada, poupando eletricidade, arrefecimento e espaço físico.
- **Snapshots e Alta Disponibilidade:** Guarde e restaure instantaneamente o estado de uma VM. As VMs podem até ser migradas entre servidores físicos sem tempo de inatividade.

O Problema: Configuração Repetitiva de VMs

Imagine que precisa de implementar 10 VMs de servidores web idênticas. O processo manual para *cada uma* seria:

1. Arrancar a VM e fazer login.
2. Definir um hostname único.
3. Configurar a rede.
4. Criar contas de utilizador e configurar chaves SSH.
5. Executar atualizações de segurança (`apt update` & `apt upgrade`).
6. Instalar o software necessário (nginx, ufw, etc.).
7. Configurar os serviços.

Isto é lento, entediante e propenso a erro humano.
Simplesmente não escala para ambientes de nuvem.

A Solução: Cloud-Init

O **Cloud-Init** é a ferramenta padrão da indústria para automatizar a **configuração inicial** de uma instância na nuvem ou máquina virtual. Foi concebido para ser executado **apenas no primeiro arranque** para provisionar o sistema.

- **Como Funciona:**

1. A plataforma de nuvem ou o hipervisor fornece dados de configuração (chamados “user data”) à VM no momento da sua criação.
2. Dentro do SO convidado, um serviço Cloud-Init arranca automaticamente no primeiro boot.
3. Este serviço encontra os “user data” e executa as instruções contidas neles para configurar o sistema.

- **Analogia:** Pense no Cloud-Init como um script de configuração automatizado que prepara o seu novo servidor antes mesmo de fazer o primeiro login.

Cloud-Init na Prática: User Data

A configuração para o Cloud-Init é tipicamente escrita em **YAML**. Este ficheiro, muitas vezes chamado `user-data`, contém um conjunto de diretivas. Com este único ficheiro, uma nova VM pode arrancar totalmente configurada sem qualquer intervenção manual.

```
#cloud-config
hostname: webserver-01

users:
  - name: admin
    sudo: ALL=(ALL) NOPASSWD:ALL
    groups: sudo
    shell: /bin/bash
    ssh_authorized_keys:
      - ssh-rsa AAAA... user@example.com

packages:
  - nginx
  - ufw

runcmd:
  - [ ufw, allow, 'WWW Full' ]
```

I/O Virtual e Rede

O Desafio do I/O e o VirtIO

Uma VM não tem hardware físico. O hipervisor tem de fornecer dispositivos virtuais.

- **Dispositivos Emulados (Lento):** O hipervisor finge ser um dispositivo de hardware real (como uma placa de rede Intel E1000). Máxima compatibilidade, mas lento.
- **Dispositivos Paravirtualizados (Rápido):** Os sistemas modernos usam **VirtIO**. O SO convidado tem um driver **virtio** especial que usa um canal padronizado e altamente eficiente para comunicar com o hipervisor para tarefas de disco e rede.

O VirtualBox suporta ambos: dispositivos emulados para máxima compatibilidade, e VirtIO (paravirtualizado) para desempenho quando o SO convidado possui os drivers.

- **Modo NAT (Padrão):** A VM partilha o endereço IP do seu anfitrião. Fácil de configurar e permite que o convidado aceda à internet, mas torna difícil que outros dispositivos na sua rede se conectem ao convidado.
- **Modo Bridge:** A VM obtém o seu próprio endereço IP na sua rede local, aparecendo como um dispositivo físico separado. Ideal para executar servidores.

Pode conceder a uma VM o controlo exclusivo de um dispositivo físico conectado ao seu anfitrião.

- **Passthrough de USB:** Dá a uma VM acesso direto a um dispositivo USB. Essencial para o desenvolvimento de sistemas embebidos, permitindo que a sua VM Debian programe diretamente uma placa **Arduino ou ESP32**.
- **Passthrough de PCI:** Atribui um dispositivo PCI físico, como uma potente **GPU**, diretamente a uma VM. Isto oferece desempenho quase nativo para tarefas exigentes como jogos ou machine learning.

Esta funcionalidade avançada requer suporte de hardware da CPU e do chipset da motherboard, especificamente da **IOMMU (Input-Output Memory Management Unit)**.

- **IOMMU da Intel:** VT-d
- **IOMMU da AMD:** AMD-Vi

A IOMMU cria um “sandbox” de memória seguro para o dispositivo, garantindo que este apenas pode aceder à memória da VM à qual está atribuído. Isto impede que o dispositivo interfira com o SO anfitrião ou outras VMs.

Oracle VirtualBox

Apresentando o VirtualBox

O VirtualBox é um hipervisor **Tipo-2 (hospedado)** que corre como uma aplicação padrão no seu SO existente. É desenvolvido pela Oracle e é gratuito e de código aberto.

- **Para quem é:** Iniciantes, estudantes e utilizadores de desktop que precisam de uma interface gráfica fácil de usar para executar VMs.
- **Principais Características:**
 - Multi-plataforma (Windows, macOS, Linux).
 - Interface gráfica amigável.
 - Guest Additions para integração perfeita.
 - Funcionalidade de snapshots fácil de usar.
 - Suporte para formatos de disco VDI, VMDK e VHD.

Instalar o VirtualBox

O processo envolve a instalação da aplicação principal e de um Extension Pack separado para funcionalidades completas.

1. **Download:** Vá à [página oficial de downloads do VirtualBox](#) e descarregue o pacote para o seu SO anfitrião. Descarregue também o **Extension Pack** (adiciona USB 2.0/3.0, encriptação de disco, arranque PXE).
2. **Instalar a Aplicação:** Execute o instalador da aplicação principal.
3. **Segurança no macOS:** No macOS, tem de ir a **Definições do Sistema > Privacidade e Segurança e Permitir** a extensão de sistema da Oracle.
4. **Instalar o Extension Pack:** Dê um duplo clique no ficheiro .vbox-extpack descarregado. O VirtualBox abrirá e guiá-lo-á na instalação.

Criar uma VM no VirtualBox

Criar uma VM é um processo simples, guiado por um assistente.

1. Clique no botão **“Novo”** para iniciar o assistente de nova VM.
2. Atribua um **nome** e selecione o **tipo de SO** (p. ex., “Debian 64-bit”).
3. Atribua **RAM** (p. ex., 2048 MB) e **núcleos de CPU** (p. ex., 2).
4. Quando solicitado um disco rígido, escolha **“Não adicionar um disco rígido virtual”** (para a nossa aula).
5. Vá a **Definições > Armazenamento** e clique no ícone do disco para anexar o seu ficheiro .vdi fornecido.
6. Selecione a VM e clique em **“Iniciar”**.

VirtualBox: Guest Additions

As **Guest Additions** são um conjunto de drivers e utilitários instalados *dentro* do SO convidado para melhorar a integração entre anfitrião e convidado.

- **Área de Transferência Partilhada:** Copiar e colar texto entre anfitrião e convidado.
- **Arrastar e Largar:** Arrastar ficheiros entre as janelas do anfitrião e do convidado.
- **Pastas Partilhadas:** Montar um diretório do anfitrião dentro do convidado para troca fácil de ficheiros.
- **Redimensionar Ecrã:** O ecrã do convidado ajusta-se automaticamente quando redimensiona a janela da VM.
- **Modo Seamless:** As janelas das aplicações do convidado aparecem diretamente no desktop do anfitrião.

Para instalar: No menu da VM, seleccione **Dispositivos > Inserir imagem de CD das Guest Additions** e depois execute o instalador dentro do convidado.

VirtualBox: Snapshots

Os snapshots capturam o **estado completo** de uma VM num ponto específico no tempo, incluindo conteúdo do disco, RAM e configuração dos dispositivos.

- **Porquê usar:**

- Guardar um estado funcional antes de fazer alterações arriscadas.
- Reverter instantaneamente se algo correr mal.
- Criar configurações ramificadas a partir de uma base comum.

- **Como usar:**

1. Abra **Máquina > Criar Snapshot** (ou pressione `Ctrl+Shift+S` / `Host+T`).
2. Dê ao snapshot um nome descritivo (p. ex., “Antes da atualização do kernel”).
3. Para restaurar: clique com o botão direito no snapshot e escolha **Restaurar**.

Dica: Os snapshots crescem em tamanho ao longo do tempo. Apague snapshots antigos que já não precisa para

O VirtualBox oferece vários modos de rede. Os dois mais comuns são:

Modo	Guest p/ Internet	Host p/ Guest	Guest p/ Guest
NAT	Sim	Via Port Forwarding	Não
Bridge	Sim	Sim (IP próprio)	Sim
Rede NAT	Sim	Via Port Forwarding	Sim
Host-Only	Não	Sim	Sim
Interna	Não	Não	Sim

Para esta aula, usamos **NAT** com port forwarding para aceder à VM via SSH.

VirtualBox: NAT Port Forwarding

No modo NAT, o convidado está escondido atrás do IP do anfitrião. O **port forwarding** permite alcançar serviços do convidado a partir do anfitrião.

Para configurar acesso SSH à sua VM:

1. Selecione a sua VM e abra **Definições > Rede > Adaptador 1**.
2. Confirme que o adaptador está definido para **NAT**.
3. Clique em **Avançadas > Reencaminhamento de Portas**.
4. Adicione uma nova regra:

Nome	Protocolo	IP Host	Porta Host	IP Guest	Porta Guest
SSH	TCP	127.0.0.1	2222	10.0.2.15	22

5. Ligue-se a partir do terminal do anfitrião:

```
$ ssh -p 2222 student@localhost
```

VirtualBox: Armazenamento e Formatos de Disco

O VirtualBox suporta vários formatos de disco virtual:

- **VDI (Virtual Disk Image):** Formato nativo do VirtualBox. Suporta alocação dinâmica e snapshots.
- **VMDK:** Formato da VMware. Útil para compatibilidade ao mover VMs entre VirtualBox e VMware.
- **VHD/VHDX:** Formato da Microsoft. Compatível com o Hyper-V.

Alocação dinâmica vs. fixa:

- **Alocação dinâmica:** O ficheiro de disco começa pequeno e cresce à medida que adiciona dados, até ao tamanho máximo. Poupa espaço em disco no anfitrião.
- **Tamanho fixo:** O espaço total do disco é alocado imediatamente. Desempenho de I/O ligeiramente superior.

Para esta aula, usamos um ficheiro `.vdi` pré-construído com alocação dinâmica.

VirtualBox: VBoxManage CLI

Para utilizadores avançados, o VirtualBox disponibiliza a ferramenta de linha de comandos VBoxManage. Expõe todas as funcionalidades disponíveis na GUI e mais.

Exemplos úteis:

```
# Listar todas as VMs registadas
$ VBoxManage list vms
```

```
# Iniciar uma VM em modo headless (sem janela GUI)
$ VBoxManage startvm "MinhaVM" --type headless
```

```
# Criar um snapshot a partir da linha de comandos
$ VBoxManage snapshot "MinhaVM" take "estado-limpo"
```

```
# Configurar uma regra de port forwarding NAT
$ VBoxManage modifyvm "MinhaVM" --natpf1 \
  "ssh,tcp,,2222,,22"
```

Isto é especialmente útil para scripting e automação.

QEMU + KVM

Apresentando o QEMU + KVM

O QEMU é um emulador de máquinas potente, e o KVM (Kernel-based Virtual Machine) é o módulo de virtualização integrado do kernel Linux. Juntos, fornecem virtualização **Tipo-1 (bare-metal)** de alto desempenho em Linux.

- **Para quem é:** Administradores de sistemas, programadores e utilizadores avançados que precisam de flexibilidade, desempenho e controlo por linha de comandos. É o motor por trás de muitas plataformas de nuvem em larga escala.
- **Principais Características:**
 - Extremamente flexível e programável (*scriptable*).
 - Pode emular uma enorme variedade de arquiteturas de CPU (ARM, MIPS, etc.).
 - Desempenho quase nativo quando usado com KVM.
 - Armazenamento avançado com `.qcow2` (snapshots, provisionamento dinâmico).

Instalar o QEMU + KVM

Em sistemas baseados em Debian/Ubuntu, a instalação é feita através do gestor de pacotes apt.

1. **Instalar Pacotes:**

```
bash $ sudo apt update $  
sudo apt install qemu-system-x86 \      kvm  
virt-manager libvirt-daemon-system
```
2. **Adicionar Utilizador a Grupos:** Terá de fazer logout e login novamente para que isto tenha efeito.

```
bash $  
sudo adduser $USER libvirt $ sudo adduser  
$USER kvm
```

O pacote `virt-manager` fornece uma ferramenta gráfica para gerir VMs QEMU/KVM.

Usar o QEMU + KVM

Embora o `virt-manager` forneça uma GUI, a linha de comandos demonstra o poder do QEMU.

1. **Criar um Disco Virtual:** O formato `.qcow2` é recomendado. Cria um disco de 20 GB que só cresce à medida que os dados são adicionados.

```
bash $  
qemu-img create -f qcow2 \  
o_meu_disco_debian.qcow2 20G
```
2. **Lançar uma VM a partir de um ISO:**

```
bash $  
qemu-system-x86_64 -enable-kvm \           -m 2048  
-hda o_meu_disco_debian.qcow2 \         -cdrom  
debian-13-netinst.iso -boot d
```

 - `-enable-kvm`: Usar KVM para aceleração de hardware.
 - `-m 2048`: Atribuir 2048 MB de RAM.
 - `-hda`: Ficheiro do disco rígido primário.
 - `-cdrom`: Anexar um ISO como CD-ROM virtual.
 - `-boot d`: Arrancar a partir da unidade de CD-ROM primeiro.

Comparação e Recursos

Comparação: VirtualBox vs. QEMU/KVM

Característica	VirtualBox	QEMU + KVM
Tipo	Tipo-2 (Hospedado)	Tipo-1 (via Kernel Linux)
Plataforma	Windows, macOS, Linux	Linux
Facilidade de Uso	Muito Elevada (GUI assistente)	Média a Baixa (CLI)
Desempenho	Bom (uso em desktop)	Excelente (quase nativo)
Flexibilidade	Boa	Muito Elevada (<i>scriptable</i>)
Melhor Para	Estudantes, utilizadores desktop	Servidores, cloud, programadores

Para esta aula, usamos o **VirtualBox** porque funciona em todos os sistemas operativos anfitriões e fornece uma interface gráfica simples para gerir VMs.

Guarde estas páginas para referência rápida.

- **VirtualBox:**
 - [Manual do Utilizador](#)
 - [Guia de Rede](#)
 - [Guest Additions](#)
- **QEMU:**
 - [Manual do Utilizador](#)
 - [Rede](#)
- **Cloud-Init:**
 - [Documentação](#)